ReefTrace software execution

USB WEBCAM EDITION / FOUR BUILDERS / IMPLEMENTATION + RESEARCH

Prepared 16 October 2026. Read this alongside the selected hardware manual and the repository README. The software now includes the camera/data pipeline, measured-flow source model, case store, operator dashboard, Chalk investigator and Photon adapter.

Evidence boundary: code, numerical/API/SDK-contract tests and browser replay were exercised. No real webcam/tank, live Chalk/Photon account, authenticated Copernicus download or reef deployment was connected. Synthetic results are not physical accuracy results.

One software path

INPUT	COMPUTATION	OUTPUT
C920s USB frames	Water-plane mapping + red-token tracking	Genuine recorded observations and gaps
Independent drift runs	Spatial drift fit + support/error checks	Immutable measured field profile
Candidate sources + permitted path/endpoint	Forward stochastic particles + conditional likelihoods	Ranked source support or explicit Unknown
Saved evidence + operator question	Bounded shared agent through Chalk; Photon ingress/egress	Grounded report, trace and manual task proposal

The existing pump remains on its own original wall cord. There is no motor/serial/gantry control in this software. The LLM does not generate source scores.

Prepared 16 October 2026 for four builders and the USB Webcam Edition 5.0 hardware. The repository has been cloned and implementation work is in `codex/software-execution`. Commands below run from the clone root. The software is a local tank prototype with explicit synthetic and measured modes; an ocean demonstrator is a separate future experiment.

What the software actually does

1. A C920s USB webcam supplies real frames to an isolated capture process. The laptop does the computation; the camera is not connected to Arduino, Wi-Fi or a model API.

2. OpenCV detects one standardized red token and records genuine centroid observations, time stamps, missing detections and track identity breaks. A measured water-plane homography converts pixels to centimetres.

3. Repeated unlabeled drift recordings fit a spatial interpolation model. Held-out drift runs check its error/support. No public dataset can replace this tank measurement.

4. A forward stochastic particle model asks which declared source polygons could have produced the permitted downstream path. For a token first seen already retained, it compares particles retained there across unknown release ages. The output can be Unknown.

5. FastAPI saves immutable evidence and owns the case/action state. The dashboard displays numerical scores directly. Chalk routes the bounded investigator, which reads those records and may run frozen sensitivity checks or save one genuine new recording. Photon carries that same investigator into the prepared direct phone conversation.

6. A human may approve and complete a manual net-recovery task. Neither the agent nor the application controls the AC pump.

Do we need machine learning?

A neural network is not needed for this controlled build. Red-token colour segmentation is fast, explainable and testable on the exact webcam. The learned numerical component is the drift field fitted from measured marker motion. The source estimator is physics-based inverse inference. The LLM is the investigation interface, not a source-label generator.

Do not train a new detector merely to have an ML slide. If the eventual target becomes real underwater debris, collect target-domain annotations, compare a pretrained detector with a simple baseline, verify dataset/weight licences, and measure detection precision/recall on untouched sessions. TrashCan is relevant underwater imagery; MARIDA is multispectral satellite data. Neither supplies this prototype's inlet labels or flow. See `datasets.md` for primary references and access limits.

Execution environment and infrastructure

Component	Selected implementation	Why it is needed
Scientific runtime	Python 3.12, NumPy, SciPy	Measured field, particles, stable likelihoods and sensitivity
Camera/geometry	OpenCV headless 4.x	Detection, mapping and JPEG evidence; preview is in browser
Local API	FastAPI + Uvicorn, one worker, loopback port 8000	Own USB device/session, typed contracts and operator authority
Persistence	SQLite WAL + local evidence frame files	Immutable results, atomic approvals, durable inbox/outbox
Operator UI	Existing Next.js/React/TypeScript scaffold, now interactive	Replay, reports, camera setup and investigation
Agent model route	Chalk Model Gateway through httpx	Shared bounded case/tool loop and evaluation
Phone channel	Official Photon Python SDK 0.5.0, explicit sponsor extra	Allowlisted direct messaging with idempotent durable replies
Cloud resources	The two sponsor accounts only for live integrations	No GPU, hosted camera server, Docker, Redis or managed DB needed
Ocean extension	Optional xarray/NetCDF4/Copernicus Toolbox	Small explicit geographic-field downloads and offline validation

Run `scripts/bootstrap.py` once; it creates a private `.env` with a random local operator token and does not print it. Enter that local token in the dashboard. Keep Chalk and Photon secrets server-side and never put them into `NEXT_PUBLIC_` variables. The browser only needs `NEXT_PUBLIC_API_BASE_URL`, defaulting to localhost:8000. Use an explicit assigned API URL if the dashboard runs on another computer; this baseline remains local and does not expose a public operator service.

Install exact successful versions from the lock snapshots once verified. Python optional extras keep ocean access and remote evaluations out of the live camera's critical path. The gateway itself needs only httpx; the full Chalk Functions evaluation runner is a separate optional extra and explicit command.

First run before hardware arrives

Follow README installation/startup. Generate the committed synthetic inputs with `python scripts/generate_demo.py`, then load the replay in the UI. Expected examples:

Synthetic observation	Intended behavior
Informative downstream trajectory	Conditional source ranking
First-seen-retained endpoint in a shared pocket	Unknown / ambiguous sources

Synthetic observation	Intended behavior
Incompatible calibration ID	Invalid calibration

Every synthetic field/trap/label is visibly computational. A fixture's `validated:true` means inside that synthetic fixture, not that a real pocket was tested. The evaluation truth file is only read by the scorer. Do not label the tank or ocean model accurate from these three cases.

`python scripts/evaluate.py` runs the same estimator before it reads separate labels for scoring. It reports attempted/ranked/unknown/invalid counts, coverage and accuracy among ranked cases separately. The 1/K random-source baseline is an analytical reference, not a trained performance result. Chalk policy evaluation and model/source-physics evaluation are distinct reports.

Physical data acquisition, in the right order

1. Assemble the selected fixed pump/hose/pocket/camera rig. Make no source-attribution claim until geometry and repeatability gates pass. Measure the actual bin; do not use the larger synthetic box dimensions.

2. Start the external C920s from the dashboard at its tested device index. Confirm the preview is the external camera. Record delivered dimensions and session ID. Verify steady lighting, privacy shutter, whole useful region and small-token visibility. Hardware was not connected during this implementation.

3. Fill `config/calibration.example.json` with at least four non-collinear pixel <-> world pairs at the water surface, plus at least three independent check pairs. Record actual image size and current session ID. POST `/api/v1/vision/calibrate` with the local token, or use the dashboard form. `approved:true` requires the measured maximum check-point error to meet the declared tolerance. Do not reuse fitting points as checks.

4. Collect repeated calibration recordings using `/api/v1/camera/record` with `{calibration_id: "actual-id", duration_s:5}`. **No source profile is required yet.** Move one standard token at a time through development grid locations, remove hands, let flow settle, and record. Do not put the release label into the API observation. Recording returns genuine raw detections and frame IDs, with explicit gaps/identity breaks.

5. GET `/api/v1/recordings/{id}/flow-samples` for nonoverlapping finite-difference drift samples. Fit on some complete runs and hold out other complete runs. POST `/api/v1/flow/fit-recordings` with disjoint `training_recording_ids` and `validation_recording_ids`, actual `bounds`, development-chosen `cell_size_cm` and `coverage_radius_cm`. It checks recording/session/calibration/frame provenance and derives the two sample sets itself. `/flow/fit` remains an explicitly unverified array-analysis endpoint; it cannot certify a measured live profile. The return is the full `flow` object for the profile.

6. Fill `config/profile.example.json` with measured bounds, that flow object, A/B/C polygons, an observation polygon that hides upstream release information, validated surface trap geometry, the actual calibration ID and token class. Set `valid:true` only after measurement/review. Freeze actual parameter values, including `max_validation_rmse_cm_s`, `revalidation_max_rmse_cm_s`, development-derived error bandwidths, release-age prior and sensitivity envelope. Templates containing nulls are intentionally not valid profiles.

7. POST `/api/v1/profiles`. Measured profiles require an approved map and separate supported flow-validation samples. The initial validation-coverage gate is 90%; it is an implementation gate, not evidence that source accuracy is90%. The independent drift RMSE must also pass the positive frozen `max_validation_rmse_cm_s` limit. The complete flow object must be the unchanged immutable result from the recorded-flow endpoint. Snapshot pump setting, waterline, hose/pocket position, camera pose and light in the run log.

8. From the dashboard select the frozen **measured** profile, record a source trial, and turn a continuous unambiguous recording into a case. The permitted observation polygon removes forbidden release positions before inference. The raw saved frame remains evidence; the estimator never receives release truth.

9. Compute the report, then inspect fit/coverage/Unknown reasons. Do not tune parameters on final-test outcomes. A source outside A/B/C or two indistinguishable sources can remain unresolved. A manually placed token at the same retained endpoint may be observationally indistinguishable from a genuine arrival.

The headless capture worker saves sampled original JPEG frames and receipt timestamps. It does not currently archive every 30 fps frame as an encoded continuous movie. Measurement cadence, packet/frame gaps and host receipt vs actual exposure timing remain visible. This is adequate for the bounded observed-segment prototype only after timing/detection tests; do not claim unrecorded intervals were observed.

API payloads and practical workflow

Use http://127.0.0.1:8000/docs for typed request bodies and supply `Authorization: Bearer <local token>` for mutations. Never paste real secrets into a public issue or sample file.

A direct replay observation accepts only the case fields documented by `CaseInput`:

```
{
  "profile_id":"actual-profile-id","calibration_id":"actual-calibration-id",
  "token_class":"red-foam-10mm","mode":"REPLAY",
  "points":[{"t_s":0,"x_cm":10,"y_cm":5,"quality":1,"observed":true},
            {"t_s":0.5,"x_cm":11,"y_cm":5.2,"quality":1,"observed":true},
            {"t_s":1,"x_cm":12,"y_cm":5.4,"quality":1,"observed":true}],
  "evidence_frame_ids":[]
}
```

Those numbers illustrate schema only. A first-seen-retained replay uses `points:[]` and an `endpoint:{x_cm,y_cm,retained:true,quality:1}` backed by actual retained evidence and validated pocket geometry. Keep release time/source in the evaluator-only ledger. Direct POST `/cases` accepts replay/synthetic data only. LIVE cases are derived from the immutable recording-to-case route, which checks association continuity and real frame provenance. For retained mode, request `observation_type: retained_endpoint`; coordinates are derived from a stable recorded token inside a validated pocket, never supplied by the caller. LIVE tasks additionally require the same current camera session, post-disturbance observations and a target observation no older than the configured 5 seconds. A fresh preview alone does not refresh old case evidence. A synthetic profile cannot be used in LIVE mode.

The core routes are profiles, calibration/flow fit, cases, estimate, sensitivity, investigate, recording, manual-task proposal/approval/outcome, rig disturbance/revalidation and STOP. Evidence is immutable. Creating a new estimate creates a new evidence ID; approvals bind the exact case revision and evidence snapshot. A recorded disappearance is not a recovered outcome.

Recovery or fixture movement invalidates current rig readiness. POST `/rig/disturbed` whenever anything moves. To resume, record fresh drift after the disturbance and POST `/rig/revalidate` with `profile_id`, `recording_id`, and four `checks:true`: `waterline_unchanged`, `pump_setting_unchanged`, `hose_and_pocket_unchanged`, `camera_and_light_unchanged`. The measured drift must be supported and below the previously frozen RMSE limit. Failed checks require recalibration, not widening the tolerance until the test passes. The new-data paths and manual task endpoints never command hardware.

Four-person ownership and handoffs

Owner	Concrete work	Handoff/evidence
A: apparatus/data	Fixed water rig, token accounting, waterline/source/pocket geometry; truth keeper for final trials	Frozen fixture log, retention tests, source ledger inaccessible to estimator
B: vision	USB permissions/index, independent homography checks, red-token detection and timestamps	Actual frames, mapping error, gap/identity logs, drift and observation recordings
C: model/science	Drift fit/coverage/holdouts, dispersion/error model, forward ensembles, identifiability and frozen gates	Versioned profile; source-blind estimates, sensitivity reports, coverage+wrong rankings+Unknown counts
D: application/integrations	API/SQLite/UI, Chalk provisioning/shared agent/evals, Photon route/inbox/outbox; capture content	Real sponsor request/response evidence after credentials; API/UI test results; public build story without secrets

Hours	Whole-team gate
0-3	Run synthetic software, prove one actual USB frame, verify real Chalk access and a prepared Photon channel
3-8	Map water surface; collect calibration/validation drift; record explicit misses, never fabricated positions
8-13	Fit field, measure path support, test whether A/B/C are distinguishable; redesign entry geometry only on development trials
13-18	Freeze profile; connect real observation -> saved estimate -> dashboard -> Chalk question; prove the Photon durable path
18-24	Test held-out cases with source hidden; evaluate actual agent questions and failure conditions; record every attempted result
24-29	Freeze code/config, run final labeled trials and compute coverage/conditional accuracy/Unknown; no tuning on those outcomes
29-33	Rehearse path/retained/invalid demo, verify actual manual recovery separately, prepare genuine replay for inspection
33-36	Pack frozen rig/profile/evidence, show software vs physical results clearly, finish public ASUS content submission

If a gate fails, cut presentation extras rather than conceal invalid data. The public data/ocean extension does not block the tank build and cannot substitute for failed tank identifiability. The hardware plan stays one USB camera with the existing independent pump.

Demonstration to judges

Start with one measured hidden-source trajectory and show the permitted path, conditional source table and evidence/profile IDs. Ask the same question in the prepared Photon conversation; show Chalk's actual tool trace and the saved report. Compare a first-seen-retained case that produces Unknown, then an invalid calibration that blocks ranking. If manual recovery is shown, approve and physically recover/count the standard token, then record the outcome and revalidation requirement. State real held-out results, attempted count and abstentions. Use the public content challenge to document the genuine build, tradeoffs and trial evidence; do not present synthetic footage as the physical experiment.

Known readiness boundaries

Automated numerical, API, persistence, agent-policy and SDK-contract tests can run here. No actual USB webcam/tank, real sponsor account, Copernicus authenticated download or reef deployment was connected during implementation. Readiness endpoints report configuration only. Exact live API access and camera/backend behavior are team commissioning gates. See `verification.md` for the commands/results actually executed; docs and mock tests do not establish live-service delivery or ecological impact.

Dataset decisions and primary sources

Own tank recordings are the required training/calibration/validation data for this apparatus. These public resources serve distinct future experiments; none identifies which inlet released the red token.

Copernicus Global Ocean Physics: hourly SMOC merged currents, roughly 0.083-degree (~9 km) grid, utotal/vtotal in m/s. Useful future transport forcing; too coarse for this tank or reef passages. Download requires a free account and explicit small bounds/time/depth. The reader keeps geographic units separate.

NOAA Global Drifter Program: actual historical buoy positions/velocities. A tiny public 24-row fixture was downloaded with units, timestamp and content hash. It tests ingestion, not plastic attribution or forecast skill.

TrashCan, University of Minnesota: annotated underwater debris imagery. Relevant to a later underwater detector, not surface drift/source truth. Publisher rights were not fully retrievable; no mirror/weight permissions assumed or weights bundled.

MARIDA, authors' Zenodo release: CC BY 4.0 multispectral Sentinel-2 patches and masks. Relevant to satellite debris confounders; unsuitable as direct RGB webcam training.

The repository docs/datasets.md records dataset IDs, DOI, access/licence evidence, exact bounded Copernicus CLI syntax, known update/coverage limits and downloaded NOAA provenance. docs/physics.md gives the equations and code contracts.

Implemented mathematics

```
dX = (u(X) + b) dt + sqrt(2D) dW
X[k+1] = X[k] + (u(X[k])+b)*dt + sqrt(2D*dt)*N(0,I)
source_support = normalize(exp(log_likelihood + log_explicit_prior))
```

u is the fitted marker-drift field, in cm/s; D is constant dispersion in cm^2/s. Source polygons seed forward ensembles. Trajectory likelihood uses temporally correlated residuals and a shared model/mapping error. Retained-endpoint likelihood counts particles actually retained at observation age, averaging over all particles and the declared age prior.

Poor absolute fit, unsupported paths, weak source separation, stale calibration, unsupported retention and sensitivity to frozen assumptions revoke ranking. More particles, a larger LLM and normalized scores cannot recover unobserved history or distinguish identical evidence.

Primary numerical references

Higham (2001), Euler-Maruyama; BAKTRAK forward inverse-search precedent; Pierard et al. (2022), marine source likelihoods. Prototype residual kernels, tolerances and gates still need actual development data.

Verification snapshot

The completed automated run passed 155 Python tests. Ruff passed. The Next.js production build and ESLint passed. Browser checks exercised saved replay reports, frozen sensitivity output and an honest unavailable-Chalk response. The real Photon 0.5.0 SDK was tested against HTTP mocks; Chalk 2.13.7 evaluation declarations were checked without deploying cloud functions.

Production npm audit reports zero known vulnerabilities in the checked dependency graph. Five high advisories remain in development-only Next ESLint transitive glob/braces tooling, with no patched braces release available at the check; avoid linting untrusted external pattern input. Python reports one Starlette/httpx TestClient deprecation warning. No live sponsor messages or paid model calls were made.

Start the implemented software

```
python3 -m venv .venv
source .venv/bin/activate
python -m pip install -e '.[dev,sponsors]'
python scripts/bootstrap.py
python scripts/generate_demo.py
python -m uvicorn apps.api.reeftrace_api.main:app --workers 1
# Second terminal, inside dashboard/: npm ci && npm run dev
```

Open http://127.0.0.1:3000 and enter the local token from the private .env. The API schema is at http://127.0.0.1:8000/docs.

Collection timing constraint

The current target-age gate is five seconds. A Chalk/Photon round trip may exceed it, so an old task must fail until the target is freshly verified. Show source investigation and uncertainty as the core demo; manual recovery is a separately verified operation. Do not extend freshness by merely refreshing the camera preview.

See the code snapshot README, docs/verification.md, work reports in the local clone, and the separate software-evaluation reports for exact scope. The synthetic fixture accuracy denominator is tiny and is not a reported physical benchmark.